

Lab 4 — The CAN-Bus and Transmission of Sensor Signals in a CAN Network

B.Sc. Information Engineering, HAW Hamburg
Bussystems and Sensors (IE6-BU/BUL) — Prof. Dr. Rasmus Rettig

Authors: Ahmed Mohamed, 2757098
Jamal Alden Shamsan, 2756372
Name Htet Wai Yan Tun, 2757519

Sommersemester 2026

Contents

1	Objective	2
2	Part 1: CAN Interface Setup and Test	2
2.1	Task Description	2
2.2	Experimental Setup	2
2.3	Program Function	4
2.4	Flowchart	6
2.5	Parameter Investigation	6
2.5.1	Reference Test	6
2.5.2	CAN ID Parameter	7
2.5.3	Extended Frame Parameter	8
2.5.4	Remote Frame Parameter	8
2.5.5	Bitrate Parameter	9
2.5.6	Data Length Parameter	9
2.5.7	Payload Data Parameter	10
2.5.8	Conclusion of Parameter Investigation	11
3	Part 2 — CAN Physical Layer (Loopback)	11
3.1	Task Description	11
3.2	Oscilloscope Configuration	11
3.3	Setup	12
3.4	Oscilloscope Capture	13
3.5	Annotated CAN Telegram	14
3.6	Bitrate Comparison	14
4	Part 3 — Sensor Setup and Transfer to CAN	15
4.1	Task Description	15
4.2	Block Diagram	16
4.3	Flow Diagram	16
4.4	Design and Code	16
4.5	Verification	18

5	Part 4 — Automatic Error Checking	19
5.1	Task (in own words)	19
5.2	Block Diagram / Flowchart	19
5.3	Design and Code	20
5.4	Measured Error Rate	21
6	Part 5: Setup of a Large CAN Network	22
6.1	Task Description	22
6.2	Flowchart of the investigation for Part 5	24
6.3	Block Diagram	25
6.4	Encoding of Sensor Data in the CAN Message	25
6.5	Annotated Program Code	26
6.6	Proof of Transmission	27
6.7	Reception From Another Group	29
7	Bibliography	31
A	Provided Source Code	32
A.1	CAN_test.py (sender)	32
A.2	CAN_listen.py (receiver)	32

1 Objective

The objective of this lab is to investigate CAN bus communication using a USB-CAN interface. First, a loopback setup is used to test basic CAN transmission between two CAN channels. Different CAN message parameters such as bitrate, CAN ID, payload, data length, remote frame, and extended frame are modified and observed.

In the next step, the physical CAN signal is measured with an oscilloscope. The differential voltage between CANH and CANL is analyzed, and the influence of different bitrates is compared.

Afterwards, a DS18B20 temperature sensor is integrated into the setup. The measured temperature values are read by Python, encoded into CAN payload bytes, transmitted over the CAN bus, and checked by comparing sent and received messages. Finally, the setup is connected to a common CAN bus with other lab groups. The goal is to verify communication in a larger CAN network using a common bitrate and separate CAN identifiers.

2 Part 1: CAN Interface Setup and Test

2.1 Task Description

The objective of this part was to verify the basic operation of the USB-CAN interface. For this purpose, a CAN loopback test was performed. CAN messages were generated in Python, transmitted through one CAN channel, and received back on the second CAN channel. The provided test program `Lab_04_CAN_test.py` was analyzed and modified by changing CAN parameters such as bitrate, identifier, frame type, data length, and payload.

2.2 Experimental Setup

The experimental setup consisted of the USB-CAN adapter, a loopback connection between both CAN channels, and the Python program running in Spyder. The CANH line of one channel was connected to the CANH line of the other channel. Similarly, CANL was connected to CANL. The termination resistors were activated.

Component	Function
USB-CAN adapter	Interface between the PC and the CAN bus.
CAN channel 1	Sending channel in the test program.
CAN channel 0	Receiving channel in the test program.
CANH-CANH connection	Connection between the CANH line of channel 1 and the CANH line of channel 0.
CANL-CANL connection	Connection between the CANL line of channel 1 and the CANL line of channel 0.
Termination resistors	Activated during the loopback test to correctly terminate the CAN bus.
Python/Spyder	Used to run the CAN test program and display the received messages.

Table 1: Experimental setup for the CAN loopback test

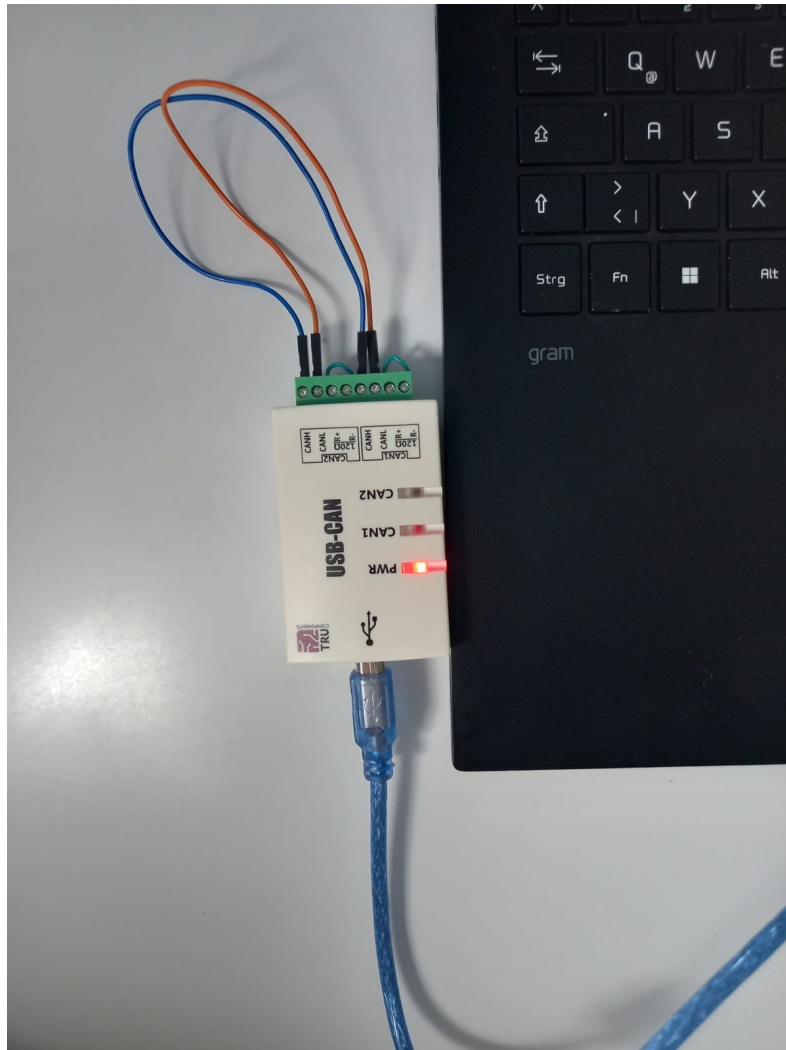


Figure 1: Photo of the CAN loopback setup

2.3 Program Function

The provided Python program was used to test the CAN communication in loopback mode. The program initializes the USB-CAN adapter, creates CAN messages with changing payload values, sends the messages on one CAN channel, and receives them on the second CAN channel.

The most important program line for initializing the CAN interface is:

```
dev = canalytstii.CanalystDevice(bitrate=500000)
```

This opens the USB-CAN device and sets the CAN bitrate to 500000 bit/s. Both CAN channels of the adapter are initialized with this bitrate.

The payload and CAN message are created inside a loop:

```
for value in range(256):
    pl = (value, value, value, value, value, value, value, value)

    new_message = canalytstii.Message(
        can_id=0x3FE,
        remote=False,
        extended=False,
        data_len=8,
        data=pl
    )

    dev.send(1, new_message)
```

The loop command `range(256)` generates the values from 0 to 255. In hexadecimal, this corresponds to the range from 0x00 to 0xFF. For every loop iteration, a new payload is created. All eight payload bytes are set to the current value of the loop variable `value`.

For example, when `value = 0`, the payload is:

```
(0, 0, 0, 0, 0, 0, 0, 0)
```

This appears in the received CAN message as:

```
Data=0000000000000000
```

When `value = 255`, the payload is:

```
(255, 255, 255, 255, 255, 255, 255, 255)
```

This appears as:

```
Data=fffffffffffffff
```

The CAN message itself is created with the function `canalytstii.Message()`. The message uses the CAN identifier 0x3FE, a standard CAN frame, no remote request, and 8 payload bytes.

The relevant parameters are:

Parameter	Meaning
<code>can_id=0x3FE</code>	CAN identifier of the transmitted message.
<code>remote=False</code>	A normal data frame is sent, not a remote request frame.
<code>extended=False</code>	A standard 11-bit CAN identifier is used.
<code>data_len=8</code>	The message contains 8 payload bytes.
<code>data=pl</code>	The payload bytes are taken from the tuple <code>pl</code> .

Table 2: CAN message parameters used in the test program

After each CAN message is created, it is sent on CAN channel 1:

```
dev.send(1, new_message)
```

The received messages are read from CAN channel 0 and printed in the console:

```
for msg in dev.receive(0):
    print(msg)
```

Therefore, the program sends CAN messages on channel 1 and receives them on channel 0. Since the payload changes from 0x00 to 0xFF, the received output can be used to verify that the transmitted and received payloads match.

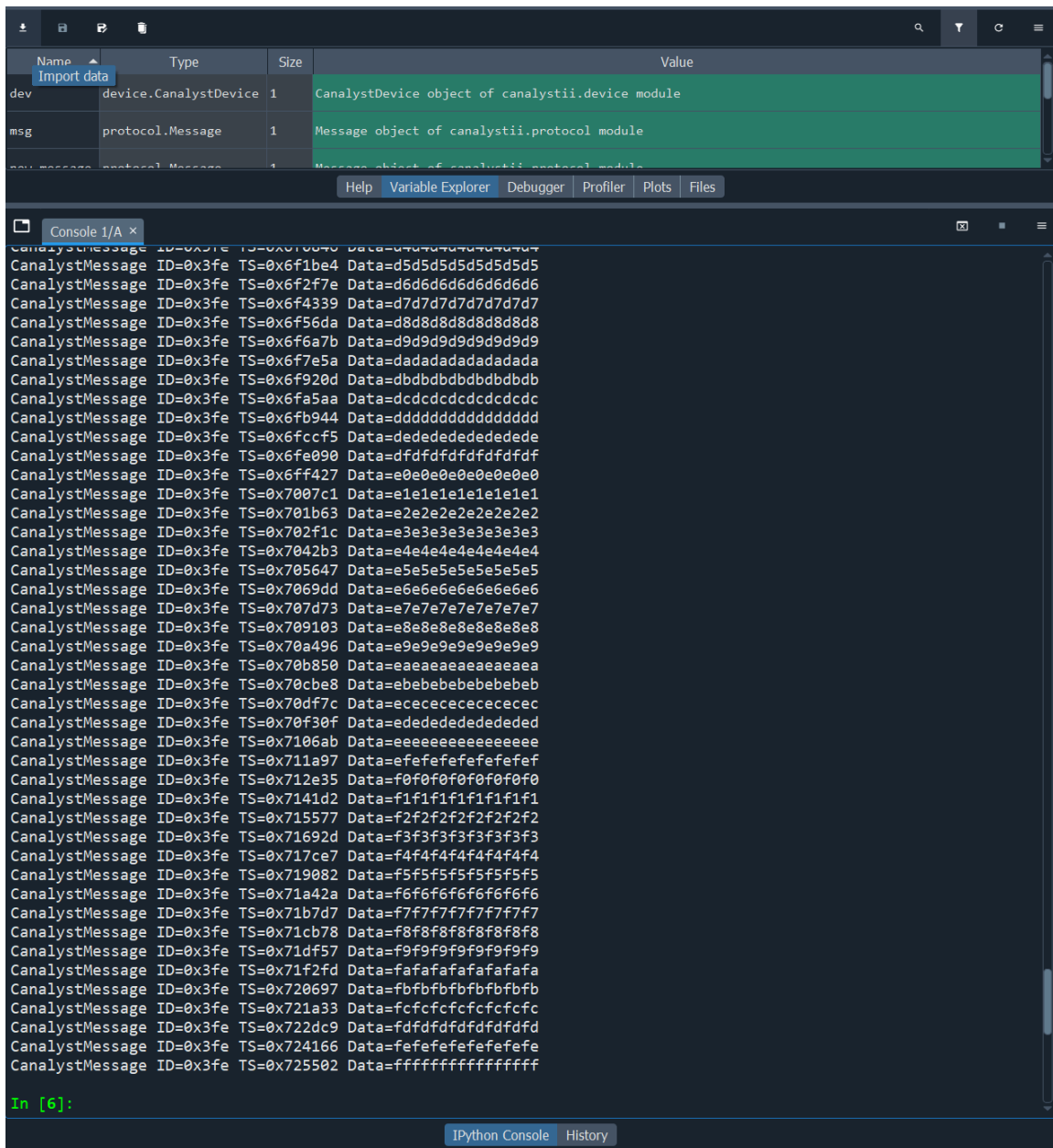


Figure 2: Console output of the CAN loopback test showing received messages with CAN ID 0x3FE and increasing payload values

2.4 Flowchart

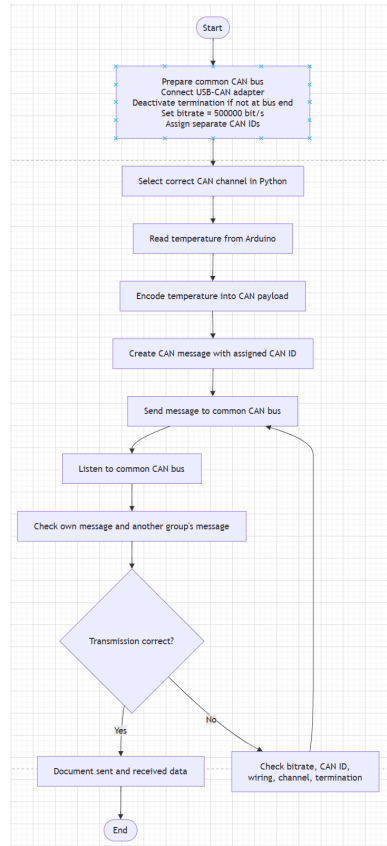


Figure 3: Flowchart

2.5 Parameter Investigation

The parameters of the CAN message were changed one at a time in order to analyze their influence on the transmitted CAN frame. For each test, only one main parameter was modified while the remaining settings were kept unchanged as far as possible.

2.5.1 Reference Test

In the reference test, the CAN message used the identifier `0x3FE`, a bitrate of `500000 bit/s`, a standard frame format, no remote request, and a payload length of 8 bytes. The payload value increased from `0x00` to `0xFF`.

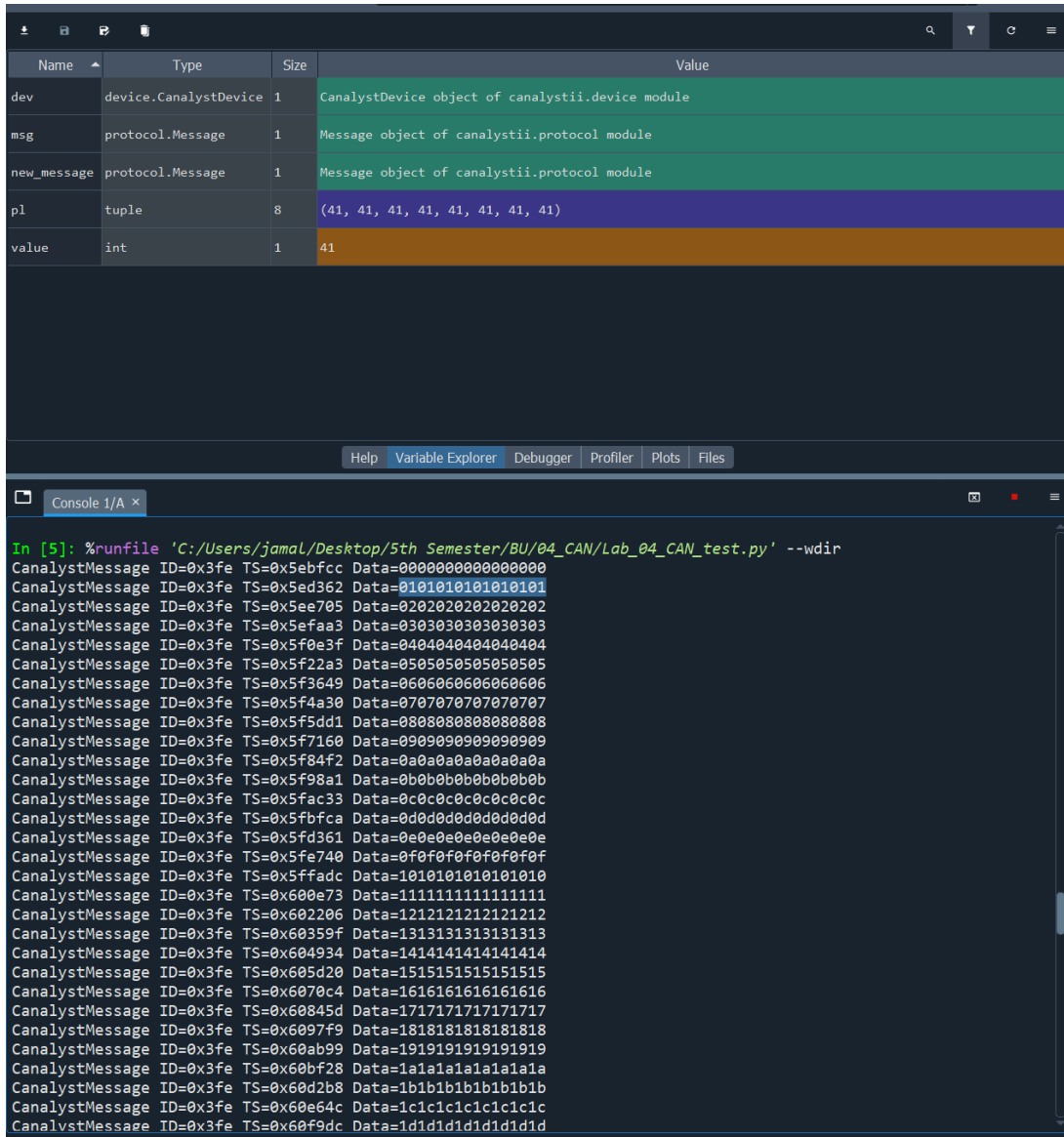


Figure 4: Reference CAN loopback test with CAN ID 0x3FE and increasing 8-byte payload values

The received messages show the expected CAN ID 0x3FE and increasing payload values. This confirms that the basic loopback communication worked correctly.

2.5.2 CAN ID Parameter

The CAN identifier was changed from 0x3FE to 0x7AA. The payload generation remained unchanged.

The image shows a Python script on the left and its console output on the right. The script is a CAN bus listener that receives messages on channel 0. It prints the received messages and then sends a copy to channel 1. The console output shows a series of received messages, all with the CAN ID 0x7AA. The messages are in the format: `CanalystMessage ID=0x3fe TS=0xc018ee Data=fbfbfbfbfbfbfbf`.

Figure 5: CAN ID parameter test showing received messages with the modified CAN ID 0x7AA

The console output shows messages with the new CAN ID 0x7AA. This confirms that the CAN identifier is defined in the message creation and can be changed by the programmer.

2.5.3 Extended Frame Parameter

The parameter `extended` was changed from `False` to `True`. In this test, an extended CAN identifier was used.

The image shows a Python script on the left and its console output on the right. The script is a CAN bus listener that receives messages on channel 0. It prints the received messages and then sends a copy to channel 1. The console output shows a series of received messages, all with the CAN ID 0x1ABCDE. The messages are in the format: `CanalystMessage ID=0x1abcde TS=0x100fca Data=2b2b2b2b2b2b2b2b`.

Figure 6: Extended frame test using `extended=True` and the extended CAN ID 0x1ABCDE

The received messages show the extended CAN ID 0x1ABCDE. This demonstrates that the adapter can transmit frames using the extended 29-bit identifier format.

2.5.4 Remote Frame Parameter

The parameter `remote` was changed from `False` to `True`. In this case, the frame is treated as a remote request frame instead of a normal data frame.


```

9 import canalystii
10 import time
11
12 # Connect to the Canalis-II device
13 # Passing a bitrate to the constructor causes both channels to be initialized and started.
14 dev = canalystii.CanalystDevice(bitrate=500000)
15
16 # Receive all pending messages on channel 0 / clear buffer
17 for msg in dev.receive(1):
18     print(msg)
19
20 # sample code for continuous listening:
21 while True:
22     msg=dev.receive(0)
23     if msg: print(msg)
24
25 # The canalystii.Message class is a ctypes Structure, to minimize overhead
26
27 for value in range(256):
28     pl=(value, value, value, value, value, value, value, value) #payload is created here
29     new_message = canalystii.Message(can_id=0x7AA,
30                                     remote=True,
31                                     extended=False,
32                                     data_len=8,
33                                     data=pl)
34
35     # Send one copy to channel 1
36     dev.send(1, new_message)
37     time.sleep(0.5)
38
39     for msg in dev.receive(0):
40         print(msg)
41
42 # Stop both channels (need to call start() again to resume capturing or send any messages)
43 dev.stop(0)
44 dev.stop(1)
45
46 # delete dev at the end to free the interface
47 del dev
48

```

```

CanalystMessage ID=0x7AA TS=0x63a89a Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x63b04d Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x63c7ff Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x63e391 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x63f730 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x6403a0 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x641e87 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x64322c Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x6445f4 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x64596b Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x646d0c Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x648043 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x649448 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x64a7e9 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x64bb58 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x64cfc3 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x64e2de Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x64f68b Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x650a2a Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x651dc8 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x65317e Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x65451d Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x6558cb Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x656c51 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x658000 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x6593ab Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x65a745 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x65ba22 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x65ce78 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x65e215 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x65f634 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x66095b Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x661cf7 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x6630a1 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x664440 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x6657d4 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x666a67 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x667f1a Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x6692c9 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x66a667 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x66ba07 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x66cd08 Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x66e15f Data=0000000000000000
CanalystMessage ID=0x7AA TS=0x66f4f4 Data=0000000000000000

```

Figure 7: Remote frame test with `remote=True`; the frame is sent as a request frame

The test shows that the frame is still transmitted with the selected CAN ID. However, a remote frame is conceptually different from a normal data frame because it is used to request data rather than to transmit sensor data directly. For the following sensor transmission tasks, `remote=False` is used because real payload data must be sent.

2.5.5 Bitrate Parameter

The bitrate was changed from 500000 bit/s to 250000 bit/s. Since both CAN channels of the adapter were initialized with the same bitrate, communication was still possible.

```

9 import canalystii
10 import time
11
12 # Connect to the Canalis-II device
13 # Passing a bitrate to the constructor causes both channels to be initialized and started.
14 dev = canalystii.CanalystDevice(bitrate=250000)
15
16 # Receive all pending messages on channel 0 / clear buffer
17 for msg in dev.receive(1):
18     print(msg)
19
20 # sample code for continuous listening:
21 while True:
22     msg=dev.receive(0)
23     if msg: print(msg)
24
25 # The canalystii.Message class is a ctypes Structure, to minimize overhead
26
27 for value in range(256):
28     pl=(value, value, value, value, value, value, value, value) #payload is created here
29     new_message = canalystii.Message(can_id=0x3FE,
30                                     remote=False,
31                                     extended=False,
32                                     data_len=8,
33                                     data=pl)
34
35     # Send one copy to channel 1
36     dev.send(1, new_message)
37     time.sleep(0.5)
38
39     for msg in dev.receive(0):
40         print(msg)
41
42 # Stop both channels (need to call start() again to resume capturing or send any messages)
43 dev.stop(0)
44 dev.stop(1)
45
46 # delete dev at the end to free the interface
47 del dev
48

```

```

CanalystMessage ID=0x3FE TS=0x725502 Data=0000000000000000
CanalystMessage ID=0x3FE TS=0x6ac9a8 Data=0101010101010101
CanalystMessage ID=0x3FE TS=0x6aced2 Data=0202020202020202
CanalystMessage ID=0x3FE TS=0x6ad011 Data=0303030303030303
CanalystMessage ID=0x3FE TS=0x6ad1b0 Data=0404040404040404
CanalystMessage ID=0x3FE TS=0x6ad285a Data=0505050505050505
CanalystMessage ID=0x3FE TS=0x6ad3bf6 Data=0606060606060606
CanalystMessage ID=0x3FE TS=0x6ad4fa2 Data=0707070707070707
CanalystMessage ID=0x3FE TS=0x6ad564a Data=0808080808080808
CanalystMessage ID=0x3FE TS=0x6ad6f6c Data=0909090909090909
CanalystMessage ID=0x3FE TS=0x6ad83cb Data=0a0a0a0a0a0a0a0a
CanalystMessage ID=0x3FE TS=0x6ad9e5c Data=0b0b0b0b0b0b0b0b
CanalystMessage ID=0x3FE TS=0x6ada1f4 Data=0c0c0c0c0c0c0c0c
CanalystMessage ID=0x3FE TS=0x6ade581 Data=0d0d0d0d0d0d0d0d
CanalystMessage ID=0x3FE TS=0x6add916 Data=0e0e0e0e0e0e0e0e
CanalystMessage ID=0x3FE TS=0x6adec0c Data=0f0f0f0f0f0f0f0f
CanalystMessage ID=0x3FE TS=0x6ae07a Data=1010101010101010
CanalystMessage ID=0x3FE TS=0x6ae1411 Data=1111111111111111
CanalystMessage ID=0x3FE TS=0x6ae2707 Data=1212121212121212
CanalystMessage ID=0x3FE TS=0x6ae3b5a Data=1313131313131313
CanalystMessage ID=0x3FE TS=0x6ae4ef3 Data=1414141414141414
CanalystMessage ID=0x3FE TS=0x6ae6295 Data=1515151515151515
CanalystMessage ID=0x3FE TS=0x6ae7630 Data=1616161616161616
CanalystMessage ID=0x3FE TS=0x6ae89d9 Data=1717171717171717
CanalystMessage ID=0x3FE TS=0x6ae9d77 Data=1818181818181818
CanalystMessage ID=0x3FE TS=0xae113 Data=1919191919191919
CanalystMessage ID=0x3FE TS=0xaeac4b9 Data=1a1a1a1a1a1a1a1a
CanalystMessage ID=0x3FE TS=0xaeae85a Data=1b1b1b1b1b1b1b1b
CanalystMessage ID=0x3FE TS=0xaeae99 Data=1c1c1c1c1c1c1c1c
CanalystMessage ID=0x3FE TS=0xaeaffa1 Data=1d1d1d1d1d1d1d1d
CanalystMessage ID=0x3FE TS=0xaeaf348 Data=1e1e1e1e1e1e1e1e
CanalystMessage ID=0x3FE TS=0xaeaf26f Data=1f1f1f1f1f1f1f1f
CanalystMessage ID=0x3FE TS=0xaeaf3af Data=2020202020202020
CanalystMessage ID=0x3FE TS=0xaeaf412 Data=2121212121212121
CanalystMessage ID=0x3FE TS=0xaeaf6209 Data=2222222222222222
CanalystMessage ID=0x3FE TS=0xaeaf765a Data=2323232323232323
CanalystMessage ID=0x3FE TS=0xaeaf89ff Data=2424242424242424
CanalystMessage ID=0x3FE TS=0xaeaf9daa Data=2525252525252525
CanalystMessage ID=0x3FE TS=0xaeaf144 Data=2626262626262626
CanalystMessage ID=0x3FE TS=0xaeaf4e4 Data=2727272727272727
CanalystMessage ID=0x3FE TS=0xaeaf6892 Data=2828282828282828
CanalystMessage ID=0x3FE TS=0xaeaf6c2c Data=2929292929292929

```

Figure 8: Bitrate test with the CAN interface set to 250000 bit/s

The console output shows that CAN messages were still received after changing the bitrate. This confirms that CAN communication works as long as sender and receiver use the same bitrate. Compared with 500000 bit/s, the transmission is slower and the bit period is longer.

2.5.6 Data Length Parameter

The payload length was changed from 8 bytes to 4 bytes by setting `data_len=4` and using a 4-byte payload.

The screenshot shows a Python script on the left and its console output on the right. The script is a CAN bus interface using the `canalystii` library. It configures a device at 500000 baud, sends a message with a 4-byte payload (0x00, 0x01, 0x02, 0x03) repeated 256 times, and then receives messages. The console output on the right shows a series of received messages, each containing the 4-byte payload followed by zeros, demonstrating that the `data_len=4` parameter correctly filters the data.

Figure 9: Data length test with `data_len=4` and a 4-byte payload

The received data field contains only the four transmitted payload bytes, followed by zeros in the displayed output. This shows that the `data_len` parameter defines how many bytes are considered as actual payload data in the CAN frame.

2.5.7 Payload Data Parameter

The payload was changed from repeated values to a fixed useful data pattern. Instead of using the loop value eight times, the payload was set to:

```
pl = (1, 2, 3, 4)
```

The screenshot shows a Python script on the left and its console output on the right. The script is similar to the previous one but uses a fixed payload `pl = (1, 2, 3, 4)` instead of repeated values. The console output on the right shows received messages where the data field contains the fixed pattern 0x00010203, followed by zeros, confirming that the `data` parameter correctly filters the useful information from the CAN frame.

Figure 10: Payload data test showing a fixed data pattern transmitted in the CAN frame

The received data shows the fixed payload pattern. This confirms that the `data` parameter contains the useful information transmitted inside the CAN frame.

2.5.8 Conclusion of Parameter Investigation

The parameter investigation showed that each CAN message parameter directly affects the transmitted frame. The CAN ID determines the identifier visible on the bus, while the payload contains the useful data. The `data_len` parameter defines the number of valid payload bytes. The `extended` parameter allows the use of longer CAN identifiers, and the `remote` parameter changes the frame from a data frame to a request frame. The bitrate must be the same for all communicating CAN nodes. Overall, the tests confirmed that the USB-CAN interface correctly transmitted and received messages for the investigated parameter settings.

3 Part 2 — CAN Physical Layer (Loopback)

3.1 Task Description

Using two matched oscilloscope probes, a differential measurement of $U_{\text{CANH}} - U_{\text{CANL}}$ is performed while `CAN_test.py` runs. A complete telegram is captured and its fields annotated. The bitrate is then changed from 500 kBd to 250 kBd and the two captures are compared.

3.2 Oscilloscope Configuration

On the Tektronix MDO3034 the differential trace is created via the Math function as the *subtraction* Ch1 – Ch2 (with Ch1 → CANH, Ch2 → CANL). Note that the scope’s `Diff()` function is the time derivative and must *not* be used here — the required “differential measurement” is plain channel subtraction. Expected trace: $\approx 0\text{ V}$ at idle (recessive) rising to $\approx 2\text{ V}$ on dominant bits.

3.3 Setup

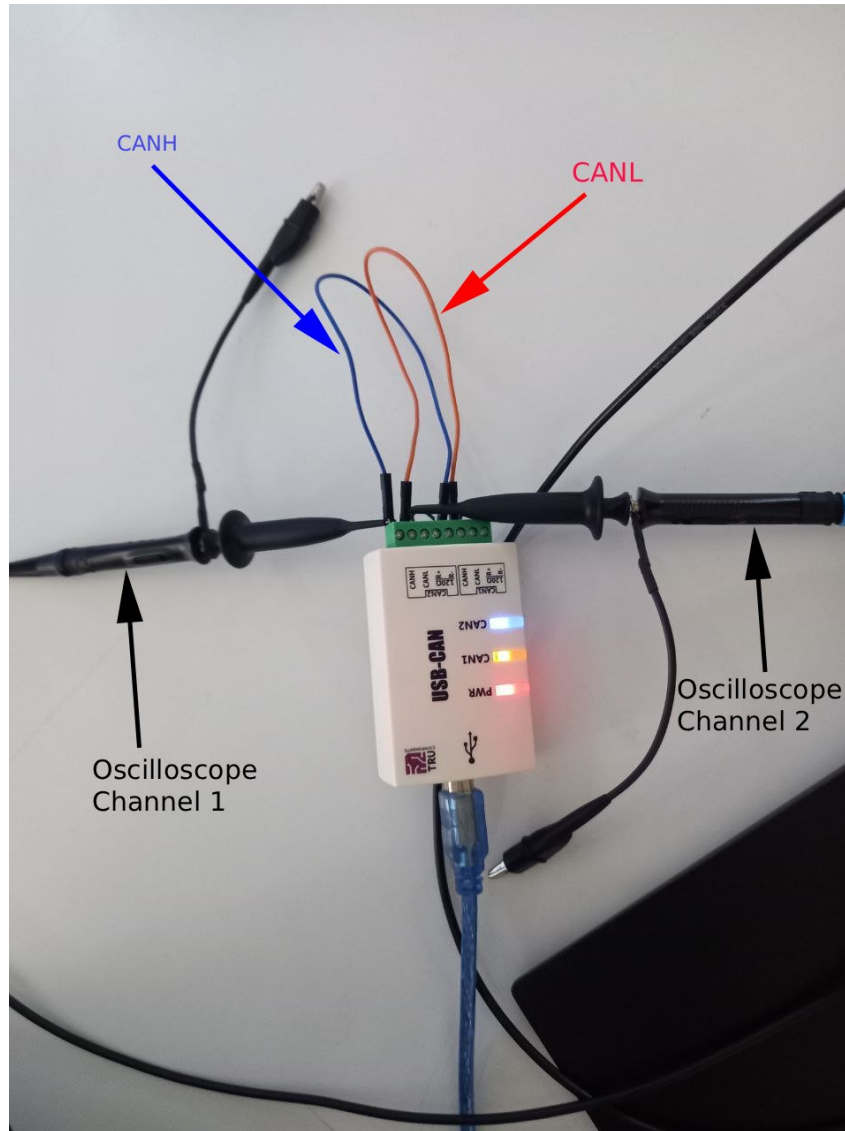


Figure 11: Experimental setup showing the USB-CAN interface

The USB-CAN adapter is connected to the CAN bus while two matched oscilloscope probes measure CANH and CANL. Channel 1 is connected to CANH and Channel 2 to CANL.

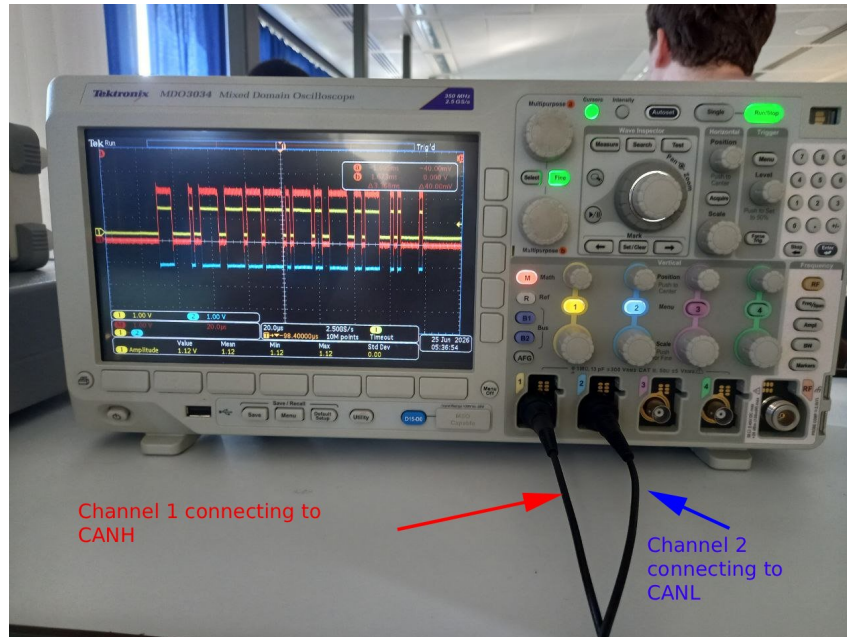


Figure 12: Image of CANH/CANL connections, and oscilloscope probes

3.4 Oscilloscope Capture

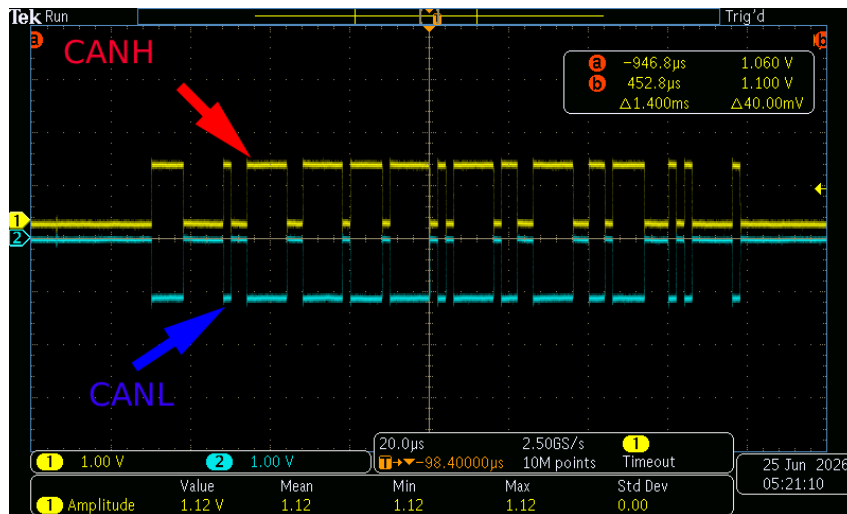


Figure 13: Oscilloscope screenshot of a complete CAN frame

The oscilloscope captures several consecutive CAN frames transmitted during the experiment. The yellow and cyan traces correspond to CANH and CANL respectively. During dominant bits the two lines move in opposite directions, producing the expected differential voltage.

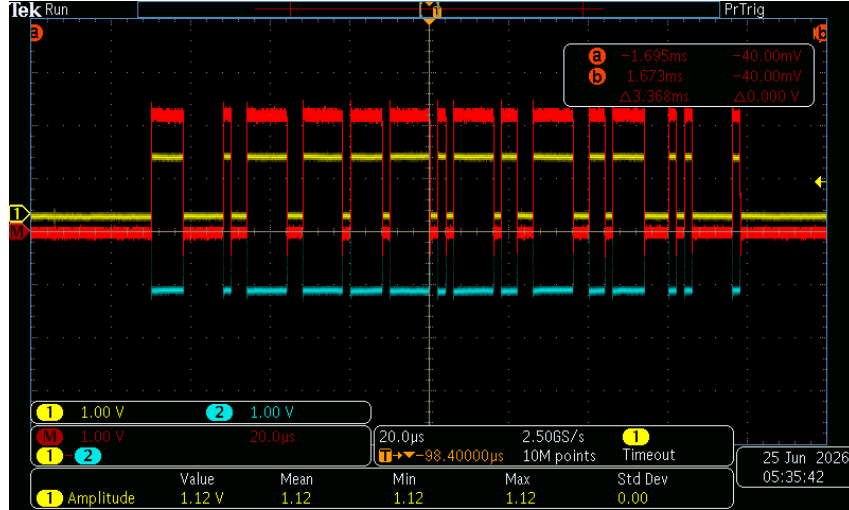


Figure 14: Differential measurement of the CANH and CANL signals during CAN communication.

3.5 Annotated CAN Telegram

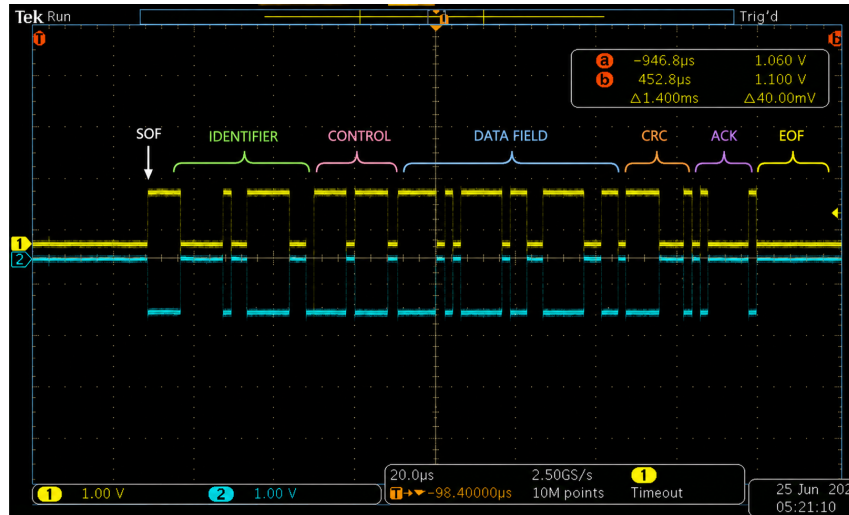


Figure 15: Figure of Annotated Can Telegram

The captured CAN telegram was annotated according to the CAN frame format. The frame begins with the Start of Frame (SOF), followed by the Identifier and Control fields. The payload is transmitted in the Data field, after which the CRC field provides error detection. The receiver acknowledges successful reception using the ACK field before the End of Frame (EOF) terminates the telegram.

3.6 Bitrate Comparison

At a bitrate of 500 kBd the measured bit time is approximately 2 μ s, whereas at 250 kBd the bit time increases to approximately 4 μ s. Consequently, each CAN frame requires roughly twice the transmission time at the lower bitrate. The voltage levels and frame structure remain unchanged; only the temporal spacing between the bits increases.

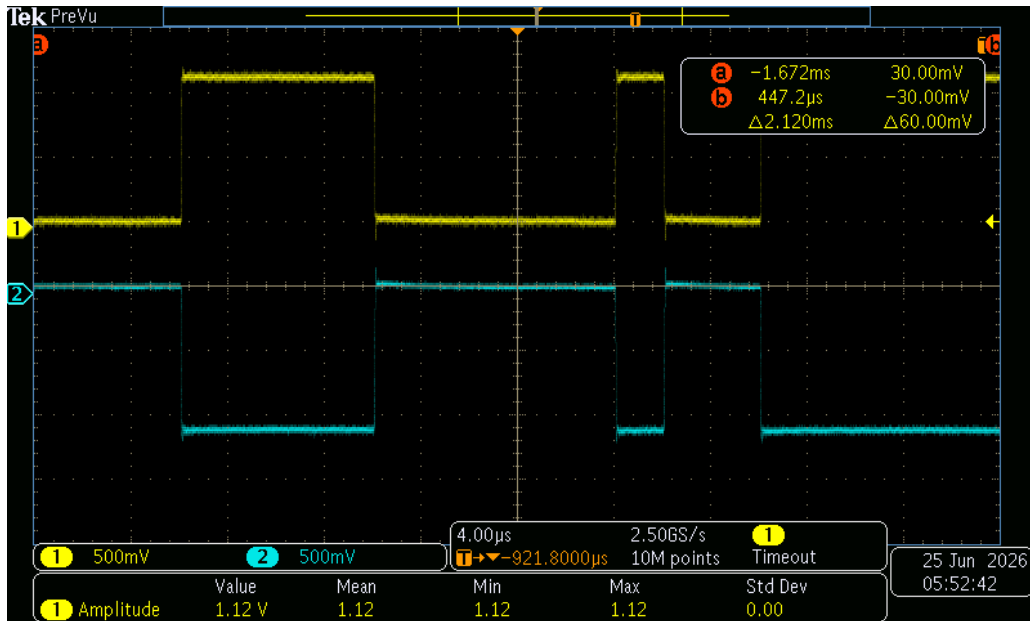


Figure 16: CAN frame at 250 kBd

4 Part 3 — Sensor Setup and Transfer to CAN

4.1 Task Description

The 1-wire temperature sensor from lab ID-S1 is set up and verified. The Python program is modified so that a sensor reading is acquired, encoded into a CAN payload, transmitted on CAN1 and read back on CAN2. The read-back value is compared with the original to confirm correct transfer.

4.2 Block Diagram

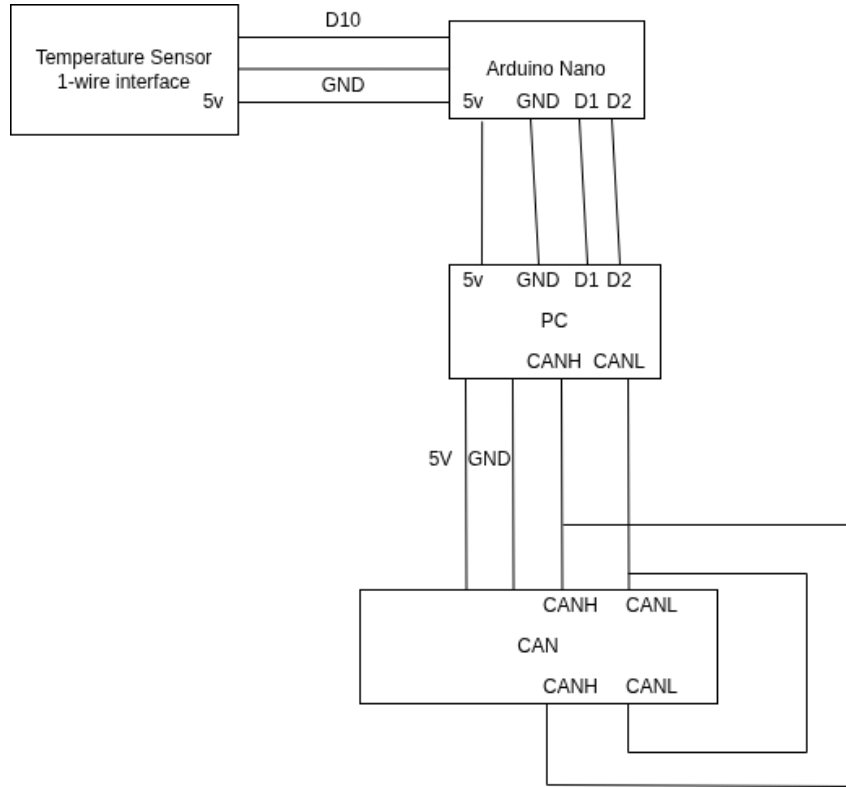


Figure 17: Hardware Block Diagram

4.3 Flow Diagram

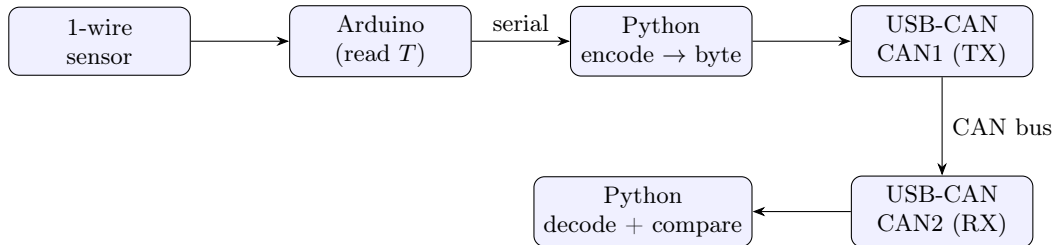


Figure 18: Data flow: sensor → Arduino → CAN1 → CAN2 → decode.

4.4 Design and Code

```

1 import serial
2 import canalytstii
3 import time
4
5 ser = serial.Serial('COM26', 19200)
6 ser.flushInput()
7
8 dev = canalytstii.CanalytstDevice(bitrate=500000)
9
10 counter = 0
11
12 while counter < 100:
13     # 1. Read temperature from Arduino
  
```



```

14 ser_bytes = ser.readline()
15 decoded = ser_bytes.decode().strip()
16 parts = decoded.split()
17 temperature = float(parts[0])
18
19 # 2. Convert temperature to integer
20 temp_scaled = int(round(temperature * 100))
21
22 # 3. Split into two bytes
23 high_byte = (temp_scaled >> 8) & 0xFF
24 low_byte = temp_scaled & 0xFF
25
26 # 4. Create payload
27 payload = (
28     high_byte,
29     low_byte,
30     1,          # sensor ID
31     counter & 0xFF, # counter
32     0, 0, 0, 0
33 )
34
35 # 5. Create CAN message
36 new_message = canaltstii.Message(
37     can_id=0x120,
38     remote=False,
39     extended=False,
40     data_len=8,
41     data=payload
42 )
43
44 # 6. Send on CAN1
45 dev.send(1, new_message)
46
47 # 7. Receive on CAN2
48 received = None
49 time.sleep(0.1)
50
51 for msg in dev.receive(0):
52     received = msg
53     print("Temperature:", temperature, " C ")
54     print("Sent payload:", payload)
55     print("Received:", msg)
56
57     counter += 1
58
59 dev.stop(0)
60 dev.stop(1)
61 ser.close()
62 del dev

```

Listing 1: Encoding a sensor byte and transmitting

The Python program is responsible for acquiring the temperature measurement from the Arduino through the serial interface, encoding the value into a CAN payload, transmitting it over CAN1, and verifying the transmission by receiving the same frame on CAN2.

The implementation consists of the following steps:

1. Establish serial communication with the Arduino at a baud rate of 19200 bps.
2. Initialize the CAN interface using the bitrate of 500 kbit/s.
3. Read one temperature value from the serial port and convert the received string into a floating-point number.

4. Multiply the temperature by 100 and convert it to an integer in order to preserve two decimal places.
5. Split the 16-bit integer into a high byte and a low byte.
6. Create an 8-byte CAN payload consisting of:
 - Byte 0: High byte of the temperature value
 - Byte 1: Low byte of the temperature value
 - Byte 2: Sensor identifier
 - Byte 3: Message counter
 - Bytes 4-7: Reserved (set to zero)
7. Create a standard CAN data frame with CAN identifier 0x120 and transmit it through CAN1.
8. Receive the CAN frame on CAN2 and compare the received payload with the transmitted payload to verify correct communication.

4.5 Verification

The screenshot shows a Python IDE with a file named 'part3.py' open. The code is a script that reads temperature data from an Arduino, converts it to a CAN payload, and transmits it via CAN1. It also receives CAN messages on CAN2 and prints the received data. The console output window on the right shows the execution of the script, displaying the sent payload and the received CAN message for several iterations.

```

5 ser = serial.Serial('COM26', 19200)
6 ser.flushInput()
7
8 dev = canaltstii.CanaltstDevice(bitrate=500000)
9
10 counter = 0
11
12 while counter < 100:
13     # 1. Read temperature from Arduino
14     ser_bytes = ser.readline()
15     decoded = ser_bytes.decode().strip()
16     parts = decoded.split()
17     temperature = float(parts[0])
18
19     # 2. Convert temperature to integer
20     temp_scaled = int(round(temperature * 100))
21
22     # 3. Split into two bytes
23     high_byte = (temp_scaled >> 8) & 0xFF
24     low_byte = temp_scaled & 0xFF
25
26     # 4. Create payload
27     payload = (
28         high_byte,
29         low_byte,
30         1,          # sensor ID
31         counter & 0xFF, # counter
32         0, 0, 0, 0
33     )
34
35     # 5. Create CAN message
36     new_message = canaltstii.Message(
37         can_id=0x120,
38         remote=False,
39         extended=False,
40         data_len=8,
41         data=payload
42     )
43
44     # 6. Send on CAN1
45     dev.send(1, new_message)
46
47     # 7. Receive on CAN2
48     received = None
49     time.sleep(0.1)
50
51     for msg in dev.receive():
52         received = msg
53         print("Temperature:", temperature, "°C")
54         print("Sent payload:", payload)
55         print("Received:", msg)
56
57     counter += 1
58
59 dev.stop(0)
60 dev.stop(1)
  
```

The console output shows the following sequence of events:

```

Sent payload: (12, 166, 1, 52, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x5ffefaf Data=0ca6013400000000
Temperature: 32.31 °C
Sent payload: (12, 159, 1, 53, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x6030b5 Data=0c9f013500000000
Temperature: 32.31 °C
Sent payload: (12, 159, 1, 54, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x6062bc Data=0c9f013600000000
Temperature: 32.31 °C
Sent payload: (12, 159, 1, 55, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x6094c4 Data=0c9f013700000000
Temperature: 32.25 °C
Sent payload: (12, 153, 1, 56, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x60c6d2 Data=0c99013800000000
Temperature: 32.25 °C
Sent payload: (12, 153, 1, 57, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x60f8d6 Data=0c99013900000000
Temperature: 32.19 °C
Sent payload: (12, 147, 1, 58, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x612aec Data=0c93013a00000000
Temperature: 32.19 °C
Sent payload: (12, 147, 1, 59, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x615cec Data=0c93013b00000000
Temperature: 32.13 °C
Sent payload: (12, 141, 1, 60, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x618f00 Data=0c8d013c00000000
Temperature: 32.13 °C
Sent payload: (12, 141, 1, 61, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x61c105 Data=0c8d013d00000000
Temperature: 32.06 °C
Sent payload: (12, 134, 1, 62, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x61f30e Data=0c86013e00000000
Temperature: 32.06 °C
Sent payload: (12, 134, 1, 63, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x622583 Data=0c86013f00000000
Temperature: 32.0 °C
Sent payload: (12, 128, 1, 64, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x625721 Data=0c80014000000000
Temperature: 32.0 °C
Sent payload: (12, 128, 1, 65, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x62892a Data=0c80014100000000
Temperature: 31.94 °C
Sent payload: (12, 122, 1, 66, 0, 0, 0, 0)
Received: CanaltstMessage ID=0x120 TS=0x62bb35 Data=0c7a014200000000
  
```

Figure 19: Output of the Screenshot showing the payload sent and the payload received

The received CAN message was compared with the transmitted payload. For example, the transmitted payload

Sent payload: (12, 159, 1, 53, 0, 0, 0, 0)

was received as

Received: Data = 0C9F013500000000

The received hexadecimal payload corresponds to the byte sequence

$(0x0C, 0x9F, 0x01, 0x35, 0x00, 0x00, 0x00, 0x00)$,

which is identical to the transmitted payload after converting the hexadecimal values to decimal:

$(12, 159, 1, 53, 0, 0, 0, 0)$.

This confirms that the CAN message was transmitted and received correctly without any data corruption. The same behaviour was observed for all transmitted messages during the experiment. For every transmitted temperature measurement, the received CAN payload matched the transmitted payload exactly. No transmission errors were observed during the loopback test.

5 Part 4 — Automatic Error Checking

5.1 Task (in own words)

The transfer from Part 3 is automated so that any deviation between the sent and received payload is reported automatically on screen. The test is run over 100 messages to determine whether messages are lost or transmission errors occur, and to compute an error rate.

5.2 Block Diagram / Flowchart

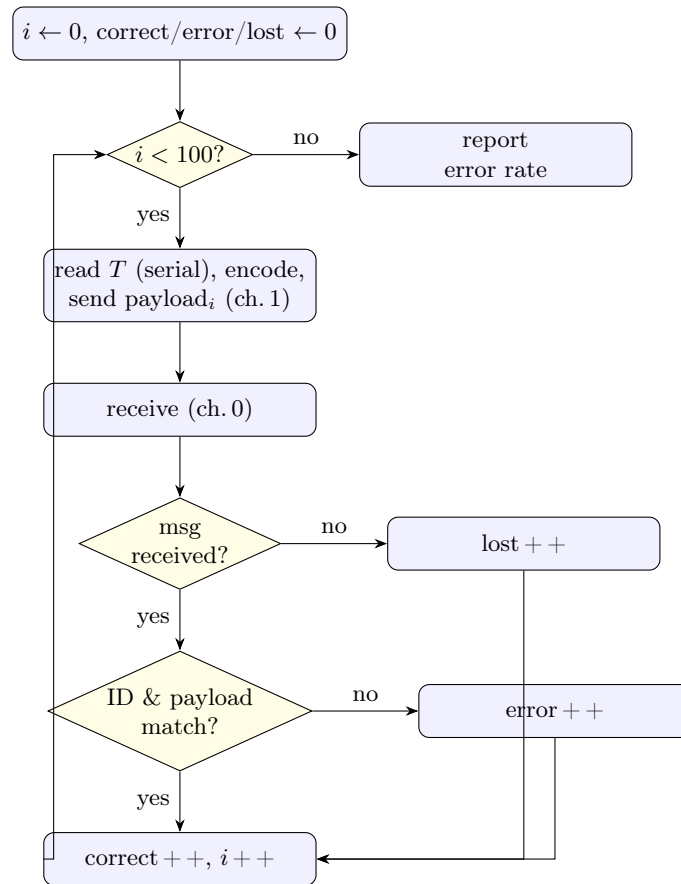


Figure 20: Automatic comparison loop over 100 messages (part4.py).

5.3 Design and Code

The payload built in Part 3 is transmitted on CAN1 and read back on CAN2. For each message the received identifier and all eight payload bytes are compared against the transmitted values. Three outcomes are counted separately: a *correct* match, an *error* (received but ID or payload differs), and a *lost* message (nothing received within the polling window). Note that this test verifies **CAN loopback transmission integrity**: the pass/fail criterion is TX-payload = RX-payload, so it does not independently validate the sensor-to-payload encoding — that is covered by Part 3.

```
1 import serial
2 import canaltstii
3 import time
4
5 # Serial connection to Arduino / temperature sensor
6 ser = serial.Serial('COM26', 19200)
7 ser.flushInput()
8
9 # CAN connection
10 dev = canaltstii.CanaltstDevice(bitrate=500000)
11
12 CAN_ID = 0x120
13 SENSOR_ID = 1
14 TOTAL_MESSAGES = 100
15
16 correct_messages = 0
17 error_messages = 0
18 lost_messages = 0
19
20 for counter in range(TOTAL_MESSAGES):
21
22     # 1. Read temperature from Arduino
23     ser_bytes = ser.readline()
24     decoded = ser_bytes.decode().strip()
25     parts = decoded.split()
26     temperature = float(parts[0])
27
28     # 2. Convert temperature to integer
29     temp_scaled = int(round(temperature * 100))
30
31     # 3. Split temperature into two bytes
32     high_byte = (temp_scaled >> 8) & 0xFF
33     low_byte = temp_scaled & 0xFF
34
35     # 4. Build CAN payload
36     sent_payload = (
37         high_byte,
38         low_byte,
39         SENSOR_ID,
40         counter & 0xFF,
41         0, 0, 0, 0
42     )
43
44     # 5. Create CAN message
45     new_message = canaltstii.Message(
46         can_id=CAN_ID,
47         remote=False,
48         extended=False,
49         data_len=8,
50         data=sent_payload
51     )
52
53     # 6. Send CAN message on channel 1
```

```

54     dev.send(1, new_message)
55
56     # 7. Try to receive message on channel 0
57     time.sleep(0.05)
58
59     received_message = None
60
61     for msg in dev.receive(0):
62         received_message = msg
63         break
64
65     # 8. Automatic check
66     if received_message is None:
67         lost_messages += 1
68         print("Message", counter, ": LOST")
69
70     else:
71         received_payload = tuple(received_message.data)
72
73         if received_message.can_id == CAN_ID and received_payload ==
sent_payload:
74             correct_messages += 1
75             print("Message", counter, ": OK",
76                   "Temperature:", temperature,
77                   "Sent:", sent_payload,
78                   "Received:", received_payload)
79         else:
80             error_messages += 1
81             print("Message", counter, ": ERROR")
82             print("Sent ID:", hex(CAN_ID), "Sent payload:", sent_payload)
83             print("Received:", received_message)
84
85     # 9. Final result
86     total_errors = error_messages + lost_messages
87     error_rate = (total_errors / TOTAL_MESSAGES) * 100
88
89     print("\n--- Final Result ---")
90     print("Sent messages:", TOTAL_MESSAGES)
91     print("Correct messages:", correct_messages)
92     print("Wrong messages:", error_messages)
93     print("Lost messages:", lost_messages)
94     print("Error rate:", error_rate, "%")
95
96     # Clean up
97     dev.stop(0)
98     dev.stop(1)
99     ser.close()
100     del dev

```

Listing 2: part4.py — automatic error checking over 100 messages.

5.4 Measured Error Rate

The test was run over 100 messages on the loopback (CAN1 → CAN2) at 500 kBd. All messages were received and matched exactly:

Table 3: Automatic error-checking result over 100 messages.

Metric	Value
Sent messages	100
Correct messages	100
Wrong messages	0
Lost messages	0
Error rate	0.0 %

```

1 import serial
2 import canyastill
3 import time
4
5 # Serial connection to / temperature sensor
6 ser = serial.Serial('/dev/ttyUSB0', 115200)
7 ser.flushInput()
8
9 # CAN connection
10 dev = canyastill.CanDevice(bitrate=500000)
11
12 CAN_ID = 0x120
13 SENSOR_ID = 1
14 TOTAL_MESSAGES = 100
15
16 correct_messages = 0
17 error_messages = 0
18 lost_messages = 0
19
20 for counter in range(TOTAL_MESSAGES):
21
22     # 1. Read temperature from /dev/rtc0
23     ser_bytes = ser.readline()
24     decoded = ser_bytes.decode().strip()
25     parts = decoded.split()
26     temperature = float(parts[0])
27
28     # 2. Convert temperature to integer
29     temp_scaled = int(round(temperature * 100))
30
31     # 3. Split temperature into two bytes
32     high_byte = (temp_scaled >> 8) & 0xFF
33     low_byte = temp_scaled & 0xFF
34
35     # 4. Build CAN payload
36     sent_payload = (
37         high_byte,
38         low_byte,
39         SENSOR_ID,
40         counter & 0xFF,
41         0, 0, 0, 0
42     )
43
44     # 5. Create CAN message
45     new_message = canyastill.Message(
46         can_id=CAN_ID,
47         remote=False,
48         extended=False,
49         data=sent_payload,
50         data_length=len(sent_payload)
51     )
52
53     # 6. Send CAN message on channel 1
54     dev.send(1, new_message)
55
56     # 7. Try to receive message on channel 0

```

```

--- Final Result ---
Sent messages: 100
Correct messages: 100
Wrong messages: 0
Lost messages: 0
Error rate: 0.0 %
In [76]:

```

Figure 21: Console output: 100/100 correct, 0% error rate.

Interpretation of the 0% result (added context). A 0% error rate is the expected outcome here, not an anomaly. With a 0.05 s pause between sends the bus carries only about 20 messages/s — far below saturation — so the RX buffer never overflows and no messages are lost. Any rare bit error on the physical layer is caught by CAN’s 15-bit CRC and corrected by automatic retransmission *below* the application layer, so it never reaches this comparison. Losses would only be expected if the inter-message pause were removed (buffer overrun) or the physical layer were degraded (missing termination, injected noise).

Bus errors vs. buffer losses (flag). The code distinguishes a *lost* message (nothing received — typically a buffer/polling miss) from an *error* (received but mismatched — a genuine data corruption). These have different causes and should not be merged into a single number.

6 Part 5: Setup of a Large CAN Network

6.1 Task Description

In this part, the private CAN loopback setup was replaced by a common CAN bus shared with other lab groups. The goal was to connect one CAN channel to the common CAN network, send our own sensor data, receive messages from other groups, and verify that communication between different participants worked correctly.

Before connecting to the common CAN bus, the termination resistors had to be deactivated. The bitrate and CAN identifiers had to be coordinated with all participating groups. A common

bitrate of 500000 bit/s was agreed on. In addition, each group used a separate CAN identifier in order to avoid conflicts on the bus.



Figure 22: Experimental Setup

6.2 Flowchart of the investigation for Part 5

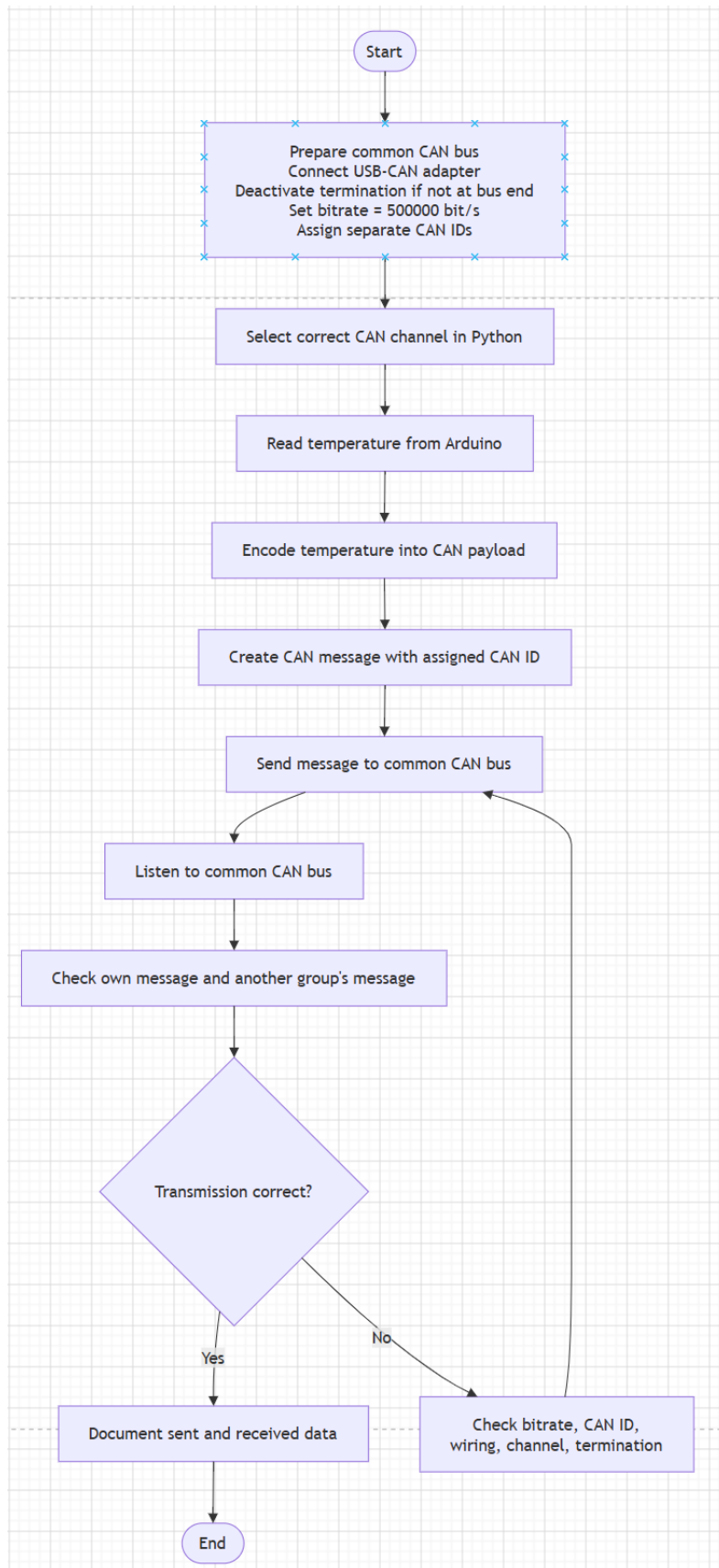


Figure 23: Flow chart of the investigation procedure for the large CAN network experiment

6.3 Block Diagram

The block diagram in Figure 24 shows the structure of the large CAN network experiment. The temperature is measured by the DS18B20 sensor and read by the Arduino. The measured value is transferred to the Python program through the USB serial interface. The Python program encodes the measurement into a CAN payload and sends it via the USB-CAN adapter to the common CAN bus, where it can be received by the other lab groups.

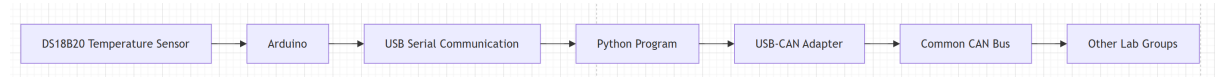


Figure 24: Block Diagram

6.4 Encoding of Sensor Data in the CAN Message

The transmitted sensor value was the temperature measured by the DS18B20 sensor. The temperature value was first read from the Arduino through the serial interface. In the Python code, the serial connection was opened on COM26 with a baud rate of 19200. The first value of each received serial line was converted into a floating-point temperature value.

Since a CAN payload consists of bytes, the measured temperature could not be sent directly as a floating-point number. Therefore, the temperature was scaled by a factor of 100 in order to keep two decimal places:

```
temp_scaled = int(round(temperature * 100))
```

For example, a measured temperature of:

32.63 C

is converted to:

32.63 * 100 = 3263

The scaled integer value was then split into two bytes. The high byte contains the upper 8 bits and the low byte contains the lower 8 bits:

```
high_byte = (temp_scaled >> 8) & 0xFF
low_byte = temp_scaled & 0xFF
```

For example:

3263 decimal = 0x0CBF

```
high_byte = 0x0C = 12
low_byte = 0xBF = 191
```

The CAN payload was created as an 8-byte tuple:

```
payload = (
    high_byte,
    low_byte,
    SENSOR_ID,
    counter & 0xFF,
    0, 0, 0, 0
)
```

The payload structure is shown in Table 4.

Byte	Meaning	Example
Byte 0	Temperature high byte	12
Byte 1	Temperature low byte	191
Byte 2	Sensor ID	1
Byte 3	Message counter	25
Byte 4	Unused	0
Byte 5	Unused	0
Byte 6	Unused	0
Byte 7	Unused	0

Table 4: Encoding of the temperature sensor value in the CAN payload

The CAN message was created using the agreed CAN identifier 0x120, a standard frame format, no remote request, and a payload length of 8 bytes:

```
new_message = canalytstii.Message(
    can_id=CAN_ID,
    remote=False,
    extended=False,
    data_len=8,
    data=payload
)
```

In this setup, CAN_ID was set to 0x120 and SENSOR_ID was set to 1. The message was then sent on CAN channel 0, which was connected to the common CAN bus:

```
dev.send(0, new_message)
```

The program also printed the transmitted CAN ID, the measured temperature, and the payload in both decimal and hexadecimal format. This made it possible to verify how the measured temperature was encoded before transmission.

6.5 Annotated Program Code

Listing 3 shows the Python code used to read the temperature value, encode it into a CAN payload, and send it to the common CAN bus.

```
1 import serial
2 import canalytstii
3 import time
4
5 # Serial connection to Arduino
6 ser = serial.Serial('COM26', 19200)
7 ser.flushInput()
8
9 # CAN device with agreed bitrate
10 dev = canalytstii.CanalytstDevice(bitrate=500000)
11
12 # Agreed CAN ID and sensor ID
13 CAN_ID = 0x120
14 SENSOR_ID = 1
```

```

15
16 counter = 0
17
18 while True:
19     # Read temperature from Arduino
20     ser_bytes = ser.readline()
21     decoded = ser_bytes.decode().strip()
22     parts = decoded.split()
23     temperature = float(parts[0])
24
25     # Scale temperature to keep two decimals
26     temp_scaled = int(round(temperature * 100))
27
28     # Split scaled value into two bytes
29     high_byte = (temp_scaled >> 8) & 0xFF
30     low_byte = temp_scaled & 0xFF
31
32     # Build 8-byte CAN payload
33     payload = (
34         high_byte,
35         low_byte,
36         SENSOR_ID,
37         counter & 0xFF,
38         0, 0, 0, 0
39     )
40
41     # Create CAN data frame
42     new_message = canalystii.Message(
43         can_id=CAN_ID,
44         remote=False,
45         extended=False,
46         data_len=8,
47         data=payload
48     )
49
50     # Send on channel connected to common bus
51     dev.send(0, new_message)
52
53     # Print data for verification
54     print("Sent CAN ID:", hex(CAN_ID))
55     print("Temperature:", temperature, "C")
56     print("Payload decimal:", payload)
57     print("Payload hex:", [hex(b) for b in payload])
58     print("-----")
59
60     counter += 1
61     time.sleep(1)

```

Listing 3: Annotated Python code for transmitting temperature data on the common CAN bus

6.6 Proof of Transmission

The transmission from our group was verified by comparing the payload sent by our sender program with the payload received on another PC. Both sides used the common bitrate 500000 bit/s and the agreed CAN ID 0x120.

```

ser = serial.Serial('COM26', 19200)
ser.flushInput()

# CAN device
dev = canalsystii.CanalystDevice(b bitrate=500000)

# Use the CAN ID agreed with the other groups
CAN_ID = 0x120
SENSOR_ID = 1
counter = 0

while True:
    # Read temperature from Arduino
    ser_bytes = ser.readline()
    decoded = ser_bytes.decode().strip()
    parts = decoded.split()
    temperature = float(parts[0])

    # Convert temperature to integer
    temp_scaled = int(round(temperature * 100))

    # Split into two bytes
    high_byte = (temp_scaled >> 8) & 0xFF
    low_byte = temp_scaled & 0xFF

    # Create payload
    payload = (
        high_byte,
        low_byte,
        SENSOR_ID,
        counter & 0xFF,
        0, 0, 0, 0
    )

    # Create CAN message
    new_message = canalsystii.Message(
        can_id=CAN_ID,
        remote=False,
        extended=False,
        data_len=8,
        data=payload
    )

    # Send on the common CAN bus
    dev.send(0, new_message)

    print("Sent CAN ID:", hex(CAN_ID))
    print("Temperature:", temperature, "°C")
    print("Payload decimal:", payload)

```

Python 3.13.9 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 19:09:58) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 9.7.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: runfile('C:/Users/jamal/Desktop/Sth Semester/BU/04_CAN/parts.py', wdir='C:/Users/jamal/Desktop/Sth Semester/BU/04_CAN/parts.py')

Sent CAN ID: 0x120
Temperature: 28.56 °C
Payload decimal: (11, 40, 1, 0, 0, 0, 0, 0)
Payload hex: ['0xb', '0x28', '0x1', '0x0', '0x0', '0x0', '0x0', '0x0']

Sent CAN ID: 0x120
Temperature: 28.5 °C
Payload decimal: (11, 34, 1, 1, 0, 0, 0, 0)
Payload hex: ['0xb', '0x22', '0x1', '0x0', '0x0', '0x0', '0x0', '0x0']

Sent CAN ID: 0x120
Temperature: 28.5 °C
Payload decimal: (11, 34, 1, 2, 0, 0, 0, 0)
Payload hex: ['0xb', '0x22', '0x1', '0x2', '0x0', '0x0', '0x0', '0x0']

Sent CAN ID: 0x120
Temperature: 28.5 °C
Payload decimal: (11, 34, 1, 3, 0, 0, 0, 0)
Payload hex: ['0xb', '0x22', '0x1', '0x3', '0x0', '0x0', '0x0', '0x0']

Sent CAN ID: 0x120
Temperature: 28.56 °C
Payload decimal: (11, 40, 1, 4, 0, 0, 0, 0)
Payload hex: ['0xb', '0x28', '0x1', '0x4', '0x0', '0x0', '0x0', '0x0']

Sent CAN ID: 0x120
Temperature: 28.5 °C
Payload decimal: (11, 34, 1, 5, 0, 0, 0, 0)
Payload hex: ['0xb', '0x22', '0x1', '0x5', '0x0', '0x0', '0x0', '0x0']

Figure 25: Our sender output showing CAN ID 0x120 and the transmitted temperature payload

```

[CanalystMessage ID=0x120 TS=0x1f52bef Data=0b28010000000000]
[CanalystMessage ID=0x120 TS=0x1f55df1 Data=0b22010100000000]
[CanalystMessage ID=0x120 TS=0x1f58ff9 Data=0b22010200000000]
[CanalystMessage ID=0x120 TS=0x1f5c1fe Data=0b22010300000000]
[CanalystMessage ID=0x120 TS=0x1f5f408 Data=0b28010400000000]
[CanalystMessage ID=0x120 TS=0x1f62613 Data=0b22010500000000]
[CanalystMessage ID=0x120 TS=0x1f65818 Data=0b22010600000000]
[CanalystMessage ID=0x120 TS=0x1f68a2d Data=0b22010700000000]
[CanalystMessage ID=0x120 TS=0x1f6bc2e Data=0b22010800000000]
[CanalystMessage ID=0x120 TS=0x1f6ee35 Data=0b22010900000000]
[CanalystMessage ID=0x120 TS=0x1f7203e Data=0b22010a00000000]
[CanalystMessage ID=0x120 TS=0x1f7524a Data=0b22010b00000000]

```

Python 3.12.4 | packaged by Anaconda, Inc. | (main, Jun 18 2024, 15:03:56) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 8.25.0 -- An enhanced Interactive Python.

In [1]: runfile('C:/Users/Student/Desktop/buslaab4/Lab_04_CAN_Listen.py', wdir='C:/Users/Student/Desktop/buslaab4/Lab_04_CAN_Listen.py')

Figure 26: Receiver output on another PC showing messages received from our CAN ID 0x120

The sent and received messages were compared using the CAN ID and payload bytes.

Item	Value
Our CAN ID	0x120
Sent payload	0b 22 01 01 00 00 00 00
Received payload	0b 22 01 01 00 00 00 00
Result	Payload identical

Table 5: Comparison between our sent payload and the payload received on another PC

The sender transmitted the payload 0b 22 01 01 00 00 00 00 with CAN ID 0x120. The receiver output shows the same CAN ID and the same data field 0b22010100000000. Therefore, the sent and received payloads are identical.

The receiver showed messages with our CAN ID 0x120. The sent and received payloads were identical, which verifies successful transmission from our group to another PC.

6.7 Reception From Another Group

The reverse direction was also checked. Another group sent a CAN message with their assigned CAN ID, and our listener program received the message from the common CAN bus.

```

Python 3.13.9 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 19:09:58) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 9.7.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: %runfile 'C:/Users/jamal/Desktop/5th Semester/BU/04_CAN/Lab_04_CAN_Listen.py' --wdir
[CanalystMessage ID=0x3fe TS=0x2142a3e Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x2145c46 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x2148e48 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x214c04b Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x214f251 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x2152455 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x2155659 Data=0b99000000000000]
[CanalystMessage ID=0x3fe TS=0x215885d Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x215ba60 Data=0b99000000000000]
[CanalystMessage ID=0x3fe TS=0x215ec64 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x2161e69 Data=0b99000000000000]
[CanalystMessage ID=0x3fe TS=0x216506b Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x216826f Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x216b472 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x216e67a Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x217187a Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x2174a7d Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x2177c85 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x217ae85 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x217e089 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x218128d Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x2184493 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x2187694 Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x218a89c Data=0b92000000000000]
[CanalystMessage ID=0x3fe TS=0x218daa0 Data=0b92000000000000]

```

Figure 27: Our listener output showing CAN messages received from another group

```

In [1]: runfile('C:/Users/Student/Desktop/buslaab4/Lab_04_CAN_test.py', wdir='C:/Users/Student/Desktop/buslaab4')
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.69 Payload: (11, 153)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.69 Payload: (11, 153)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.69 Payload: (11, 153)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)
Sent ID: 0x3fe Temp: 29.62 Payload: (11, 146)

```

Figure 28: Sender output from the other group showing the transmitted CAN ID and payload

The sent and received messages were compared using the CAN ID and payload bytes. The received payload must be identical to the payload sent by the other group. This verifies communication in the opposite direction.

Item	Value
Other group's CAN ID	0x3FE
Sent payload by other group	(11, 146)
Received payload by our listener	0b92000000000000
Decoded received payload	0b 92 00 00 00 00 00 00
Result	Payload agrees in the first two transmitted bytes

Table 6: Comparison between the other group's sent payload and our received payload

The other group sent the payload (11, 146). In hexadecimal, this is 0b 92. Our listener received data beginning with 0b 92, followed by unused zero bytes. Therefore, the received data agrees with the payload sent by the other group.

7 Bibliography

References

- [1] Prof. Dr. Rasmus Rettig, *Lecture #10 – Bussystems and Sensors*, HAW Hamburg, Summer Semester 2026.
- [2] Prof. Dr. Rasmus Rettig, *Lecture #11 – Bussystems and Sensors*, HAW Hamburg, Summer Semester 2026.
- [3] TRU Components, *USB-CAN Interface TC-9474804*, *USB-CAN Bus Converter, Datasheet*, Conrad Electronic SE. Available: <https://asset.conrad.com/media10/add/160267/c1/-/en/002368701DS00/datenblatt-2368701-tru-components-tc-9474804-can-umsetzer-usb-can-bus-sub-d9-nicht-galvan.pdf>

A Provided Source Code

A.1 CAN_test.py (sender)

```
1 # -*- coding: utf-8 -*-
2 # @author: Rasmus Rettig
3 # sample code for bus systems lab
4 # pip install canalytstii
5
6 import canalytstii
7 import time
8
9 # Connect to the Canalytst-II device
10 # Passing a bitrate to the constructor initializes and starts both channels.
11 dev = canalytstii.CanalytstDevice(bitrate=500000)
12
13 # Receive all pending messages on channel 1 / clear buffer
14 for msg in dev.receive(1):
15     print(msg)
16
17 # The canalytstii.Message class is a ctypes Structure, to minimize overhead
18 for value in range(256):
19     pl = (value, value, value, value, value, value, value, value)
20     new_message = canalytstii.Message(can_id=0x3FE,
21                                     remote=False,
22                                     extended=False,
23                                     data_len=8,
24                                     data=pl)
25
26     # Send one copy to channel 1
27     dev.send(1, new_message)
28     time.sleep(0.5)
29
30     for msg in dev.receive(0):
31         print(msg)
32
33 # Stop both channels (call start() again to resume capturing or sending)
34 dev.stop(0)
35 dev.stop(1)
36
37 # delete dev at the end to free the interface
38 del dev
```

Listing 4: CAN_test.py — provided sender program.

A.2 CAN_listen.py (receiver)

```
1 # -*- coding: utf-8 -*-
2 # @author: Rasmus Rettig
3 # sample code for bus systems lab
4 # pip install canalytstii
5 # pip install "python-can[canalytstii]"
6
7 import canalytstii
8
9 # Connect to the Canalytst-II device
10 # Passing a bitrate to the constructor initializes and starts both channels.
11 dev = canalytstii.CanalytstDevice(bitrate=500000)
12
13 # Receive all pending messages on channel 0
14 while True:
15     msg = dev.receive(0)
16     if msg:
```



```
17 | print(msg)
```

Listing 5: CAN_listen.py — provided listener program.